LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Cloud Storage Tier - Hypothesis-Based Testing

Name: Muruli

SRN: PES1UG24CS676

1. Step 1 - Dependency Installation

The practical was completed in Windows PowerShell. Python and the required testing packages were checked before running the application.

```
python --version
python -m pip install hypothesis pytest

Environment recorded: Python 3.13.7 | Hypothesis 6.170.14 | Pytest 9.2.0
```

2. Step 2 - Original Application Execution

The unmodified processor.py file was run with three valid sample inputs: 500MB, 1024:256, and HOT_TIER. The application returned 0.50 GB, a compression ratio of 4.0, and a rate of $0.08 per GB.

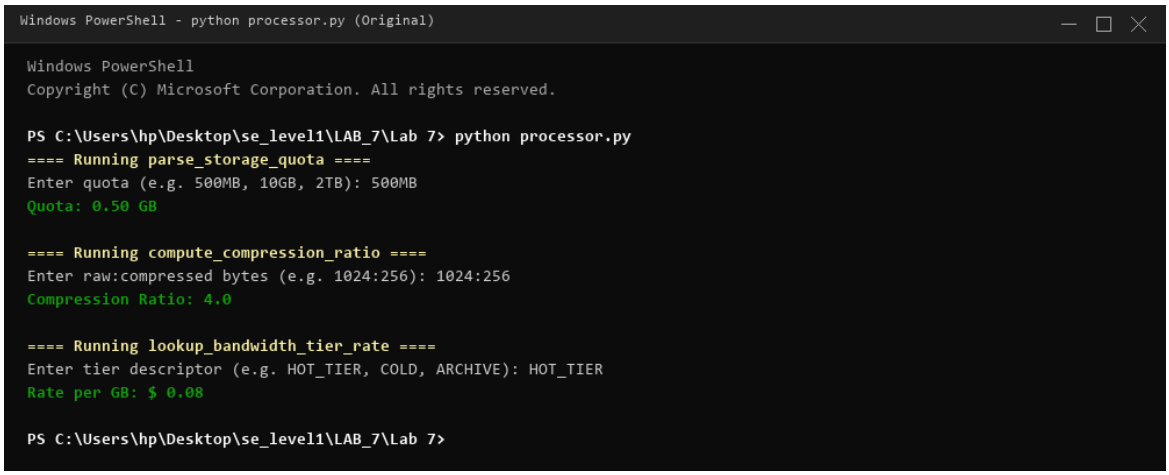


Figure 1. Original application run with valid sample inputs.

3. Step 3 - Hypothesis Test Implementation

The starter test file contained three incomplete tests. Each test was completed to exercise one function with generated text and None values. The try/except block reports an unexpected exception as a test failure.

```
from hypothesis import given, strategies as st
from processor import (parse_storage_quota,
   compute_compression_ratio, lookup_bandwidth_tier_rate)

@given(st.text() | st.none())
def test_parse_storage_quota_no_crash(s):
   try:
      parse_storage_quota(s)
   except Exception as e:
      assert False, f"parse_storage_quota crashed: {e}"

# The same pattern was used for the other two functions.
```

4. Step 4 - Initial Fuzzing Failures

The first run of python -m pytest test_processor.py -v failed in all three tests. Hypothesis reduced the generated inputs to small examples that made the problems easier to identify.

Target function	Examples found and observed problem
parse_storage_quota()	Empty input caused a numeric conversion error; None caused an attribute error; '000' reached an unsupported unit lookup.
compute_compression_ratio()	Missing or extra separators caused unpacking errors; '0:0' caused division by zero; None caused an attribute error.
lookup_bandwidth_tier_rate()	Empty or unsupported prefixes caused key errors; None caused an attribute error.

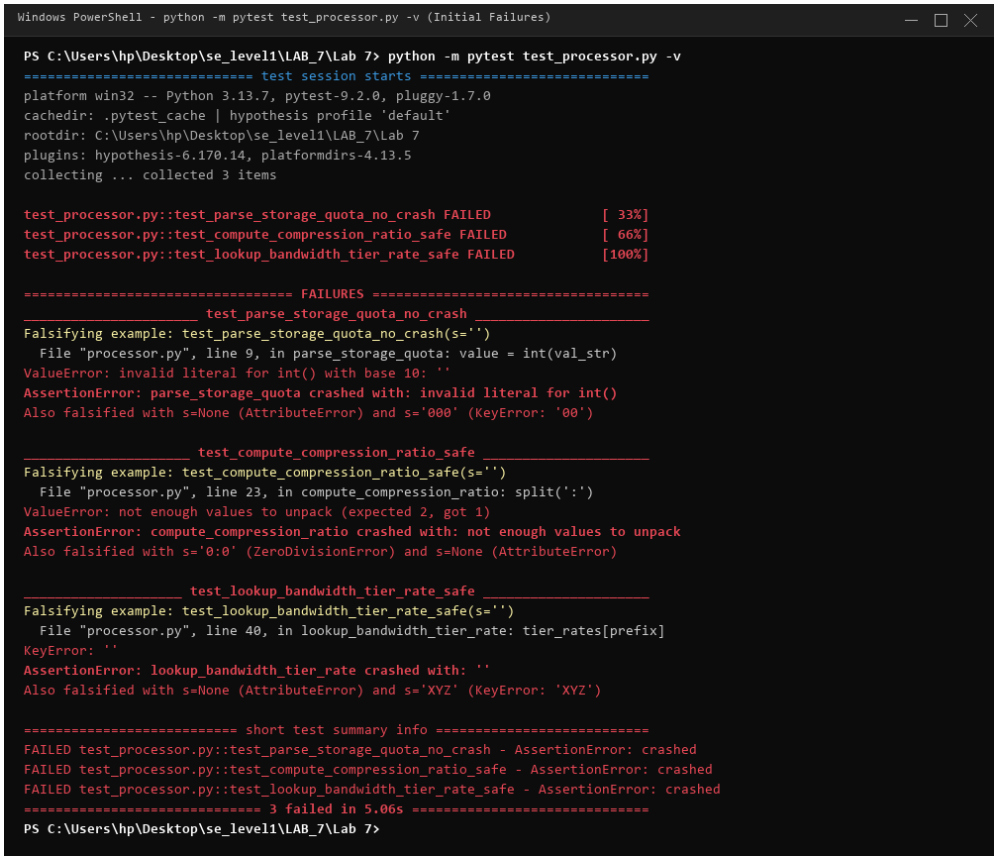


Figure 2. Initial Pytest run showing the three Hypothesis tests failing.

5. Step 5 - Bugs Identified and Corrections

The functions were updated to validate inputs before processing them. The changes preserve the supported units, tier names, and valid input formats. The interactive main() routine was also adjusted to show validation messages instead of an unhandled traceback.

Function	Correction made
parse_storage_quota()	Check the input type and length, allow only MB, GB, and TB, validate the number, reject negative and non-finite values, and return a clear validation error.
compute_compression_ratio()	Check the input type and require exactly one colon. Validate both numbers, reject negative and non-finite values, and prevent a zero denominator.
lookup_bandwidth_tier_rate()	Check the input type and minimum length, normalize the prefix, accept only HOT, COL, and ARC, and report unknown tiers clearly.

6. Step 6 - Fixed Application Execution

The corrected application was run again using the same valid inputs: 500MB, 1024:256, and HOT_TIER. The expected outputs were preserved.

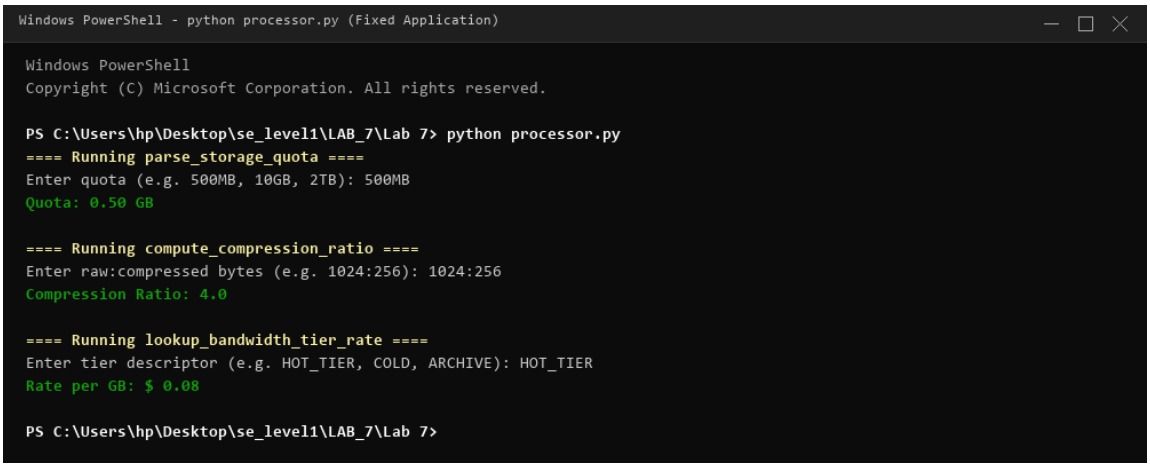


Figure 3. Fixed application run with the original valid sample inputs.

7. Step 7 - Final Test Results

The test file was updated to check generated inputs, valid results, expected validation errors, and numeric properties. The recorded final run completed with 12 tests passed and no failures.

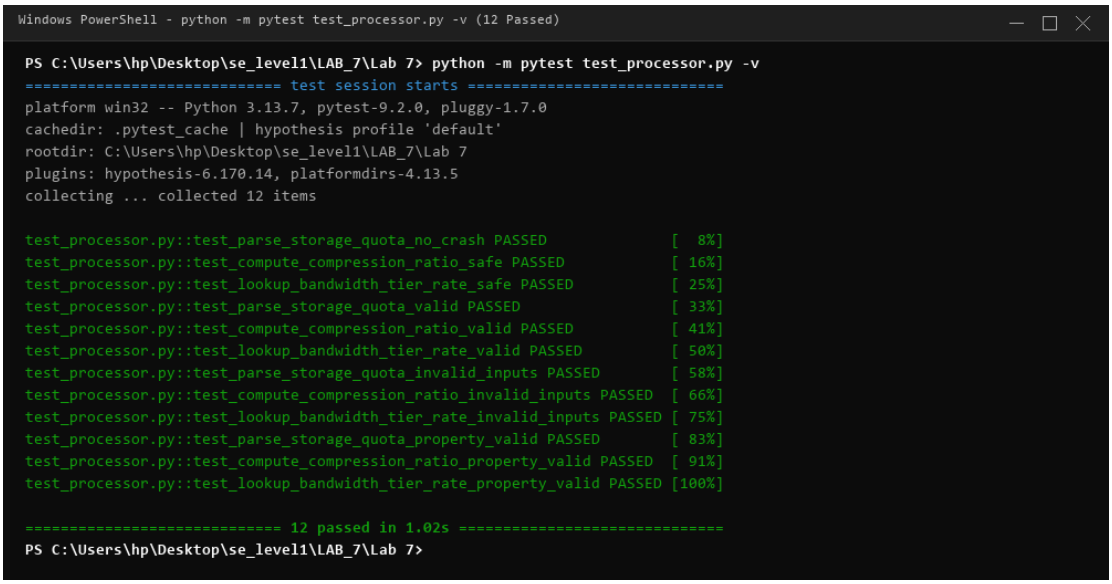


Figure 4. Final Pytest run showing 12 tests passing.

Conclusion

This practical showed how fuzz testing can find input problems that are not revealed by normal sample values. Hypothesis found failures involving empty strings, None values, invalid formats, unsupported tiers, and division by zero. Input checks and clearer error handling were added to the three functions. In the final recorded run, all 12 tests passed.

Final Deliverables

- processor.py - corrected application.
- test_processor.py - updated Hypothesis and Pytest tests.
- Lab7_Report.pdf - this report with the four original colour screenshots.